Practical No 06

A) Create a simple website for a College Information Portal and convert it into a Progressive Application.

Project Name: CollegePortalPWA

Project Type: MVC Application

Program Code:

I.Filename: Index.cshtml

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to xyz College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BCA</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
```

II.Filename: manifest.json

```
{
  "name": "College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

III.Filename: service-worker.js

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});

self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

IV.Filename: Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - CollegePortalPWA3</title>
    <script type="importmap"></script>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
    <link rel="stylesheet" href="~/CollegePortalPWA3.styles.css" asp-append-version="true" />
    <link rel="manifest" href="~/manifest.json" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom
box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-
action="Index">CollegePortalPWA3</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target=".navbar-collapse" aria-controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-
action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-
action="Privacy">Privacy</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>

    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>

    <footer class="border-top footer text-muted">
        <div class="container">
            &copy; 2026 - CollegePortalPWA3 - <a asp-area="" asp-controller="Home" asp-
action="Privacy">Privacy</a>
        </div>
    </footer>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)

    <script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then((registration) => {
                    console.log('Service Worker Registered!', registration);
                })
                .catch((error) => {
                    console.log('Service Worker Registration Failed:', error);
                });
```

```
        }
    </script>
</body>
</html>
```

V.Filename: Program.cs

```
Adding a line – app.UseStaticFiles();
```

B) Create a PWA that displays a custom offline page when the user is not connected to the internet.

Project Name: OfflinePWA

Project Type: MVC Application

Program Code:

I.Filename: index.cshtml

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to XYZ College</h3>
    <p>This website supports Offline Mode.</p>
</div>
```

II.Filename: offline.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline</title>
</head>
<body>
    <h1>No Internet Connection</h1>
    <h3>Please check your internet connection.</h3>
</body>
</html>
```

III.Filename: manifest.json

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

IV.Filename: Service-Worker

```
const CACHE_NAME = "offline-cache-v1";
const urlsToCache = [
```

```
    "/",
    "/offline.html"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
            .catch(() => {
                return caches.match("/offline.html");
            })
    );
});
```

V.Filename: Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - OfflinePWA</title>
    <script type="importmap"></script>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
    <link rel="stylesheet" href="~/OfflinePWA.styles.css" asp-append-version="true" />
    <link rel="manifest" href="~/manifest.json" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-
bottom box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-
action="Index">OfflinePWA</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target=".navbar-collapse" aria-controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-
action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-
action="Privacy">Privacy</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>

    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
```

```
    </div>

    <footer class="border-top footer text-muted">
        <div class="container">
            &copy; 2026 - OfflinePWA - <a asp-area="" asp-controller="Home" asp-
action="Privacy">Privacy</a>
        </div>
    </footer>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)

    <script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then(() => console.log('Service Worker Registered'));
        }
    </script>
</body>
</html>
```

Practical No: 07

A)Develop a PWA that works in offline mode using service Worker.

Project Name: OfflinePWA

Project Type: MVC App

Program Code

I.Filename: Index.cshtml

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>PWA Offline Demo</h3>
    <p>This application works without internet connection.</p>
</div>
```

II.Filename: manifest.json

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

III.Filename: service-worker.js

```
const CACHE_NAME = "pwa-cache-v1";
```

```
const urlsToCache = [
    "/",
    "/Home/Index"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

IV.Filename: Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>@ViewData["Title"] - _SWOfflinePWA_</title>

    <script type="importmap"></script>

    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
    <link rel="stylesheet" href="~/_SWOfflinePWA_.styles.css" asp-append-
version="true" />
```

```
    <!-- PWA Manifest -->
    <link rel="manifest" href="/manifest.json">
</head>

<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-
white border-bottom box-shadow mb-3">
            <div class="container-fluid">

                <a class="navbar-brand"
                   asp-area=""
                   asp-controller="Home"
                   asp-action="Index">
                    _SWOfflinePWA_
                </a>

                <button class="navbar-toggler"
                        type="button"
                        data-bs-toggle="collapse"
                        data-bs-target=".navbar-collapse"
                        aria-controls="navbarSupportedContent"
                        aria-expanded="false"
                        aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>

                <div class="navbar-collapse collapse d-sm-inline-flex justify-
content-between">
                    <ul class="navbar-nav flex-grow-1">

                        <li class="nav-item">
                            <a class="nav-link text-dark"
                               asp-area=""
                               asp-controller="Home"
                               asp-action="Index">
                                Home
```

```
                        </a>
                    </li>

                    <li class="nav-item">
                        <a class="nav-link text-dark"
                           asp-area=""
                           asp-controller="Home"
                           asp-action="Privacy">
                            Privacy
                        </a>
                    </li>

                </ul>
            </div>
        </div>
    </nav>
</header>

<div class="container">
    <main role="main" class="pb-3">
        @RenderBody()
    </main>
</div>

<footer class="border-top footer text-muted">
    <div class="container">
        &copy; 2026 - _SWOfflinePWA_ -
        <a asp-area=""
           asp-controller="Home"
           asp-action="Privacy">
            Privacy
        </a>
    </div>
</footer>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
```

```
    <script src="~/js/site.js" asp-append-version="true"></script>


    @await RenderSectionAsync("Scripts", required: false)


    <!-- Register Service Worker -->
    <script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then(() => console.log('Service Worker Registered'))
                .catch(error => console.error('Service Worker Registration
Failed:', error));
        }
    </script>
</body>
</html>
```

B)Create a Student Record Management PWA.

Project Name: StudentRecordPWA

Project Type: MVC App

Program Code

I.Filename: Index.cshtml

```
@model IEnumerable<StudentRecordPWA.Models.Students>


<h2>Student Records</h2>


<table border="1" cellpadding="10">
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Course</th>
        <th>Age</th>
    </tr>

    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Name</td>
            <td>@item.Course</td>
            <td>@item.Age</td>
        </tr>
    }
</table>
```

II.Filename: manifest.json

```
{
  "name": "Student Record PWA",
  "short_name": "StudentPWA",
  "start_url": "/Students",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
```

```
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

III.Filename: service-wroker.js

```
const CACHE_NAME = "student-pwa-cache";


const urlsToCache = [
    "/Students"
];


self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});


self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

IV.Filename: Layout.cshtml

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />


    <title>@ViewData["Title"] - StudentRecordPWA</title>


    <script type="importmap"></script>


    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />

    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />

    <link rel="stylesheet" href="~/StudentRecordPWA.styles.css" asp-append-
version="true" />


    <!-- PWA Manifest -->

    <link rel="manifest" href="/manifest.json">

</head>


<body>

    <header>

        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-
white border-bottom box-shadow mb-3">

            <div class="container-fluid">


                <a class="navbar-brand"

                   asp-area=""

                   asp-controller="Home"

                   asp-action="Index">

                    StudentRecordPWA

                </a>


                <button class="navbar-toggler"

                        type="button"

                        data-bs-toggle="collapse"

                        data-bs-target=".navbar-collapse"

                        aria-controls="navbarSupportedContent"

                        aria-expanded="false"
```

```
                        aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>

                <div class="navbar-collapse collapse d-sm-inline-flex justify-
content-between">
                    <ul class="navbar-nav flex-grow-1">

                        <li class="nav-item">
                            <a class="nav-link text-dark"
                               asp-area=""
                               asp-controller="Home"
                               asp-action="Index">
                                Home
                            </a>
                        </li>

                        <li class="nav-item">
                            <a class="nav-link text-dark"
                               asp-area=""
                               asp-controller="Home"
                               asp-action="Privacy">
                                Privacy
                            </a>
                        </li>

                    </ul>
                </div>
            </div>
        </nav>
    </header>

    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>

    <footer class="border-top footer text-muted">
```

```
        <div class="container">
            &copy; 2026 - StudentRecordPWA -
            <a asp-area=""
               asp-controller="Home"
               asp-action="Privacy">
                Privacy
            </a>
        </div>
    </footer>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>

    @await RenderSectionAsync("Scripts", required: false)

    <!-- Register Service Worker -->
    <script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then(() => console.log('Service Worker Registered'))
                .catch(error =>
                    console.error('Service Worker Registration Failed:', error)
                );
        }
    </script>
</body>
</html>
```

V.Filename: StudentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentRecordPWA.Models;
using System.Collections.Generic;

namespace StudentRecordPWA.Controllers
{
    public class StudentsController : Controller
    {
        public IActionResult Index()
```

```
        {
            List<Students> students = new List<Students>()
            {
                new Students
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },

                new Students
                {
                    Id = 2,
                    Name = "Amit",
                    Course = "BCA",
                    Age = 21
                },

                new Students
                {
                    Id = 3,
                    Name = "Sneha",
                    Course = "BSc CS",
                    Age = 19
                }
            };

            return View(students);
        }
    }
}
```

V.Filename: Students.cs

```
namespace StudentRecordPWA.Models
{
    public class Students
    {
```

```
        public int Id { get; set; }


        public string Name { get; set; }


        public string Course { get; set; }


        public int Age { get; set; }
    }
}
```

Practical No 08

A) Create a PWA Application with an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json

Project Name: BasicPWA

Project Type: ASP.NET Core Web App (Model-View-Controller)

Program Code:

I. Filename: Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();
app.Run();
```

II. Filename: index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Basic PWA</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="manifest" href="manifest.json">
</head>
<body>
<h1>My Progressive Web Application</h1>
<p>This is the starting page of PWA.</p>
</body>
</html>
```

III. Filename: Manifest.json

```
{
"name": "Basic Progressive Web Application",
"short_name": "BasicPWA",
"description": "Simple PWA Example",
"start_url": "/index.html",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#0d6efd",
"orientation": "portrait",
"icons": [
{
"src": "icons/icon3.png",
"sizes": "128x128",
"type": "image/png"
},
{
"src": "icons/icon4.png",
"sizes": "128x128",
"type": "image/png"
}
]
}
```

B)Create a simple Web API that returns student data in JSON format.

Project Name: StudentAPI

Project Type: ASP.NET Core Web API`

Program Code:

I.Filename: Student.cs

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
        public int Age { get; set; }
    }
}
```

II.Filename: StudentsController.cs

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

namespace StudentAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        [HttpGet]
        public List<Students> GetStudents()
        {
            return new List<Students>()
            {
                new Students
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },
                new Students
                {
                    Id = 2,
                    Name = "Sneha",
                    Course = "BCA",
                    Age = 21
                }
            };
        }
    }
```

B) Create an API endpoint to calculate Factorial of a Number and test it using Postman..

Project Name: FactorialAPI

Project Type: ASP.NET Core Web API

Program Code:

I. Filename: MathController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult Factorial(int number)
        {
            if (number < 0)
            {
                return BadRequest(new
                {
                    message = "Factorial is not defined for negative numbers."
                });
            }

            int factorial = 1;

            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }

            return Ok(new
            {
                number = number,
                factorial = factorial
            });
        }
    }
}
```

II. **Filename: lunchSetting.cs**

```
    {
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5293"
    },
    "https": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
```

```
      "applicationUrl": "https://localhost:7103;http://localhost:5293"
    }
  }
}
```

Practical No 09

A) Develop CRUD operations using ASP.NET Core Web API and Entity Framework core.

Project Name: StudentAPIWithEF

Project Type: ASP.NET Core Web API

Program Code:

I. Filename: Student.cs

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public DateTime CreatedDate { get; set; }
    }
}
```

II. Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }
        public DbSet<Student> Students { get; set; }
    }
}
```

III. Filename: StudentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public IActionResult GetStudents()
        {
```

```
            return Ok(_context.Students.ToList());
        }

        // GET BY ID
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
        {
            var student = _context.Students.Find(id);

            if (student == null)
                return NotFound();

            return Ok(student);
        }

        // INSERT
        [HttpPost]
        public IActionResult AddStudent(Student student)
        {
            _context.Students.Add(student);
            _context.SaveChanges();

            return Ok(student);
        }

        // UPDATE
        [HttpPut("{id}")]
        public IActionResult UpdateStudent(int id, Student student)
        {
            var data = _context.Students.Find(id);

            if (data == null)
                return NotFound();

            data.Name = student.Name;
            data.Age = student.Age;
            data.Course = student.Course;

            _context.SaveChanges();

            return Ok(data);
        }

        // DELETE
        [HttpDelete("{id}")]
        public IActionResult DeleteStudent(int id)
        {
            var student = _context.Students.Find(id);

            if (student == null)
                return NotFound();

            _context.Students.Remove(student);
            _context.SaveChanges();

            return Ok();
        }
    }
}
```

IV. Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;
namespace StudentAPIWithEF.Data
{
public class AppDbContext : DbContext
{
public AppDbContext(DbContextOptions<AppDbContext> options)
```

```
: base(options) { }
public DbSet<Student> Students { get; set; }
}
}
```

V. Filename: Appsetting.json

```
{
"ConnectionStrings": {
"DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;TrustServer
Certificate=True;"
},
"Logging": {
"LogLevel": {
"Default": "Information",
"Microsoft.AspNetCore": "Warning"
}
},
"AllowedHosts": "*"
}
```

VI. Filename: Apsetting.json/repos

```
{
"ConnectionStrings": {
"DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True;
"
},
"Logging": {
"LogLevel": {
"Default": "Information",
"Microsoft.AspNetCore": "Warning"
}
},
"AllowedHosts": "*"

}
```

VII.Filename : `lunchSetting.cs`

```
    {
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5293"
    },
    "https": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
```

```
      "applicationUrl": "https://localhost:7103;http://localhost:5293"
    }
  }
}
```

Practical No 10

A) Create a PWA with a web App Manifest file including:
□ App name
□ Icons
□ Start URL
□ Display mode
□ Implement Push Notifications in a PWA.

Project Name:PushNotification

Project Type: ASP.NET Core Empty Web App

Program Code:

I.Filename: Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
</head>
<body>

    <h1>My Progressive Web App</h1>

    <button onclick="showNotification()">
        Show Notification
    </button>

    <script src="app.js"></script>

</body>
</html>
```

II.Filename: manifest.json

```
{
    "name": "My PWA Application",
    "short_name": "MyPWA",
    "start_url": "/index.html",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#0d6efd",
    "icons": [
        {
            "src": "icons/ppp1.png",
            "sizes": "256x256",
            "type": "image/png"
        },
        {
            "src": "icons/ppp2.png",
            "sizes": "512x512",
            "type": "image/png"
        }
    ]
}
```

III.Filename: Service-Worker,js

```
self.addEventListener('install', event => {
    console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
    console.log('Service Worker Activated');
});
```

IV.Filename: App.js

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

function showNotification() {
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/ppp1.png'
                });
            });
        }
    });
}
```

V.Filename: Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();

app.UseStaticFiles();

app.Run();
```

B) E-Commerce PWA with Product Listing and Offline Cart To develop an E-Commerce Progressive Web Application with product listing and offline cart functionality using Service Worker and local storage.

Project Name: ECommercePWA

Project Type: ASP.NET Core Empty Web App

Program Code:

I.Filename: Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>E-Commerce PWA</title>

    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">

    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }

        .container {
            display: flex;
            gap: 30px;
        }

        .cart-section {
            width: 30%;
            border: 1px solid gray;
            padding: 15px;
            min-height: 300px;
        }

        .product-section {
            width: 70%;
        }

        .products {
            display: flex;
            flex-wrap: wrap;
            gap: 15px;
        }

        .product {
            border: 1px solid gray;
            padding: 10px;
            width: 200px;
        }

        button {
            padding: 5px 10px;
            cursor: pointer;
        }
    </style>
</head>

<body>

    <h1>E-Commerce PWA</h1>
```

```
    <div class="container">

        <div class="cart-section">
            <h2>Cart Items</h2>
            <ul id="cartList"></ul>
        </div>

        <div class="product-section">
            <h2>Products</h2>

            <div class="products">

                <div class="product">
                    <h3>Laptop</h3>
                    <p>Price: ₹ 50000</p>
                    <button onclick="addToCart('Laptop')">
                        Add to Cart
                    </button>
                </div>

                <div class="product">
                    <h3>Mobile</h3>
                    <p>Price: ₹ 20000</p>
                    <button onclick="addToCart('Mobile')">
                        Add to Cart
                    </button>
                </div>

                <div class="product">
                    <h3>Headphones</h3>
                    <p>Price: ₹ 3000</p>
                    <button onclick="addToCart('Headphones')">
                        Add to Cart
                    </button>
                </div>

            </div>
        </div>

    </div>

    <script src="app.js"></script>

</body>
</html>
```

II.Filename: manifest.json

```
{
    "name": "E-Commerce PWA",
    "short_name": "ShopPWA",
    "start_url": "/index.html",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#0d6efd",
    "icons": [
        {
            "src": "icons/ppp1.png",
            "sizes": "256x256",
            "type": "image/png"
        },
        {
            "src": "icons/ppp2.png",
            "sizes": "512x512",
            "type": "image/png"
        }
```

```
    ]
}
```

III.Filename: Service-Worker,js

```
const CACHE_NAME = 'ecommerce-cache-v1';

const urlsToCache = [
    '/',
    '/index.html',
    '/app.js',
    '/manifest.json'
];

self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

IV.Filename: App.js

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

let cart = JSON.parse(localStorage.getItem('cart')) || [];

displayCart();

function addToCart(product) {
    cart.push(product);
    localStorage.setItem('cart', JSON.stringify(cart));
    displayCart();
}

function displayCart() {
    let cartList = document.getElementById('cartList');

    cartList.innerHTML = "";

    cart.forEach(item => {
        let li = document.createElement('li');
        li.textContent = item;
        cartList.appendChild(li);
    });
}
```

V.Filename: Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();

app.UseStaticFiles();

app.Run();
```